TaskVine / VineReduce / vine-cms-analysis-stack

Module 0 - Intro

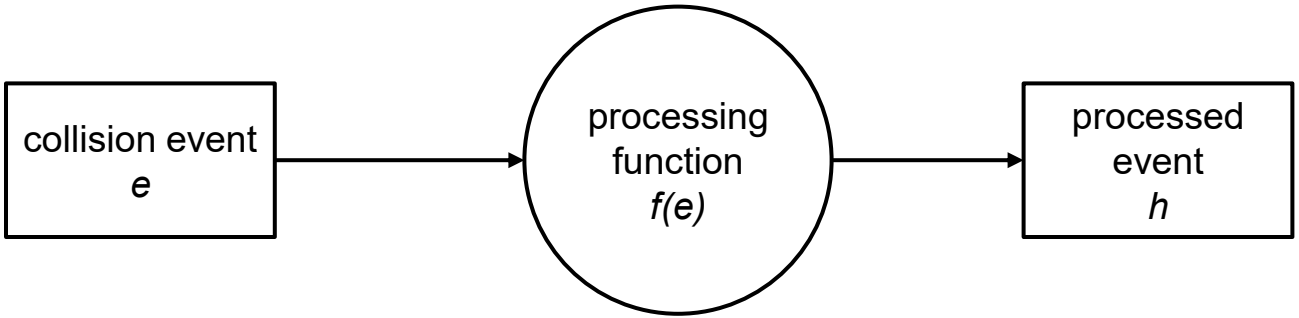
3-hour hands-on tutorial for HEP graduate students

Scaling from First Principles

Apply processing function **f** to event **e** to get result **h**.

f: E → H

h = f(e)

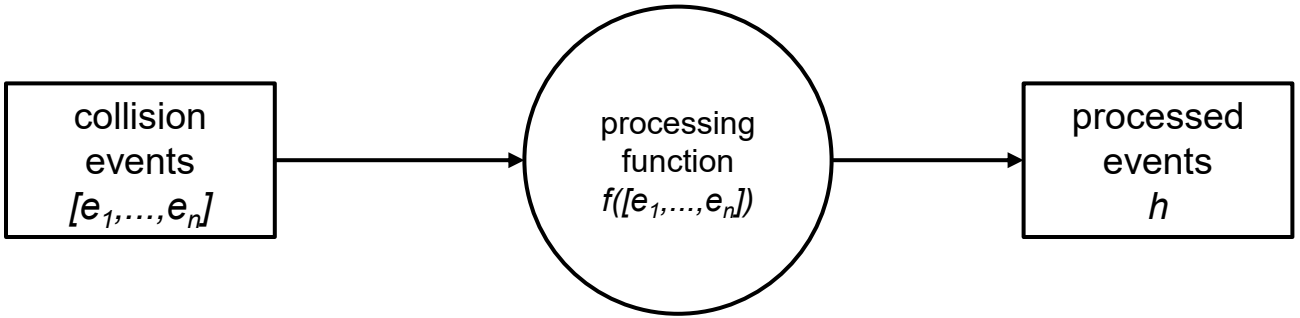


Scaling from First Principles

Almost always we process more than one event per function call.

$f: E \rightarrow H, \quad +: H \times H \rightarrow H$

$f([e_1,...,e_n]) = f(e_1) + ... + f(e_n)$



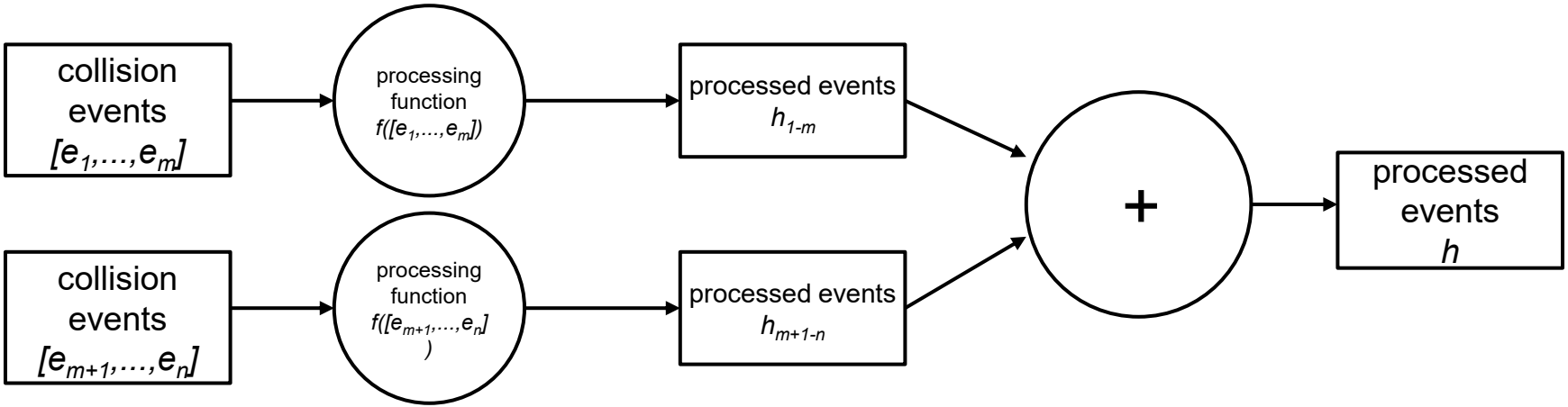
If $f([e_1,...,e_n])$ can run efficiently in a single machine, we are done.

Scaling from First Principles

The + only comes into play when we want to scale the computation, and only if it is associative.

(It is like map-reduce, but the reduce part is only important when it allows us to scale.)

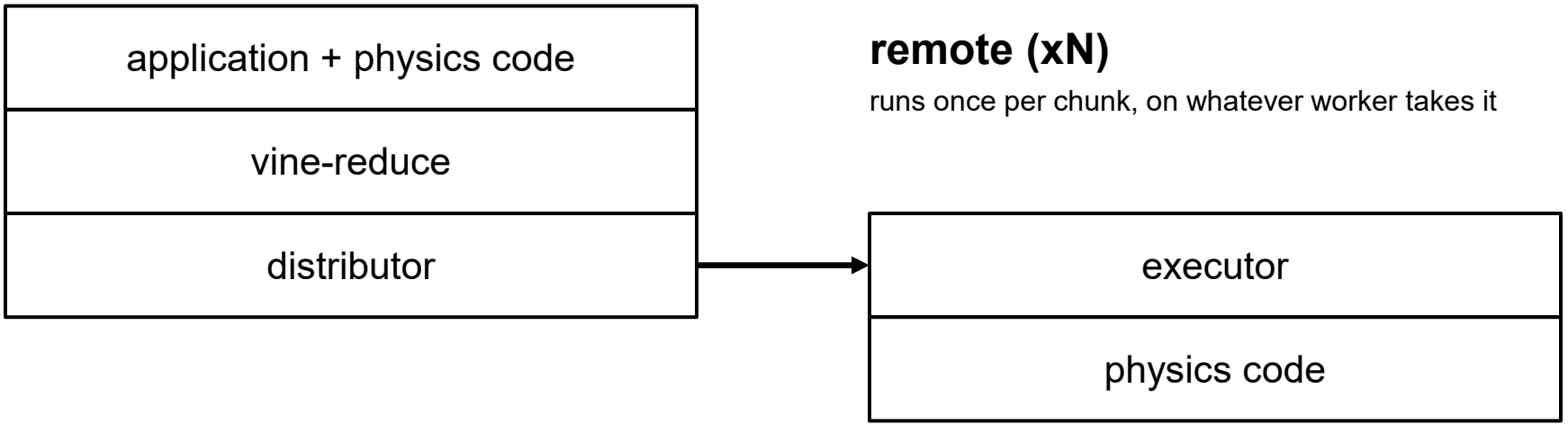
$f([e_1,...,e_n]) = f([e_1,...,e_m]) + f([e_{m+1},...,e_n]) = f(e_1) + ... + f(e_n)$



The shape of the stack

local (x1)

runs once, on the machine you are logged into



What we are running

- ADL benchmark Q6: a top-quark trijet selection on CMS Open Data
- Small, but the same shape as a real
- Written in coffea or RDataFrame; you pick one



Partial results are added together until one result per dataset remains.

How datasets and files are defined

```
{
  "SingleMu_Run2012B_193834-194270": {
    "metadata": { "dataset": "...", "source": "..." },
    "files": {
      "/project01/.../file_0.root": {
        "object_path": "Events",
        "num_entries": 53733
      },
      "/project01/.../file_1.root": {
        "object_path": "Events",
        "num_entries": 222177
      },
      ...
    }
  },
  "SingleMu_Run2012B_194271-194789": { ... },
  ...
}
```

Dataset name: one entry per dataset; each is its own pipeline with its own result

files: absolute paths (or root:// URLs); one dataset = many files

object_path: the tree to read inside each file

num_entries: events in the file; the total and the chunk boundaries come from here

Excerpt of data/datasets_vast.json. This is the output of coffea's preprocess(); vine_reduce reads it but does not create it (only num_entries is needed per file). Eventually vine_reduce will preprocess directly. [*How to preprocess*]

Why VineReduce, alongside coffea/RDataFrame

Parallelism

Owned by VineReduce

- Split a dataset into chunks
- Map the processor over chunks
- Reduce outputs back together

Distribution

Handed to a distributor (today: TaskVine)

- Schedule chunks onto machines
- Transfer code and data

Execution

Owned by coffea/RDataFrame

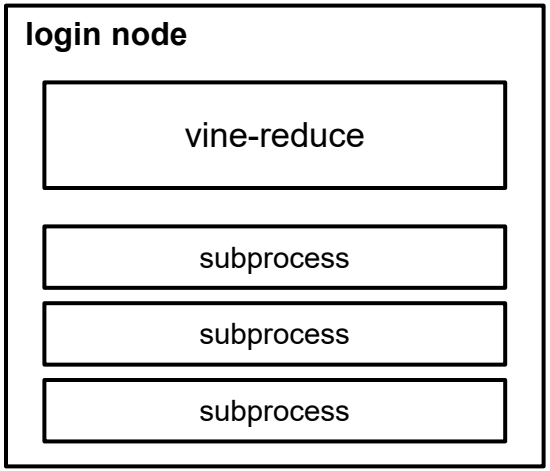
- Execute code on remote workers

The analysis code never changes. Only what it is handed to does.

Three places for execution

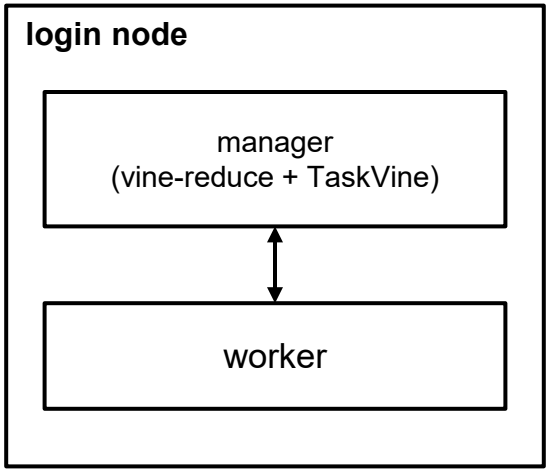
1 Local distributor

Module 2



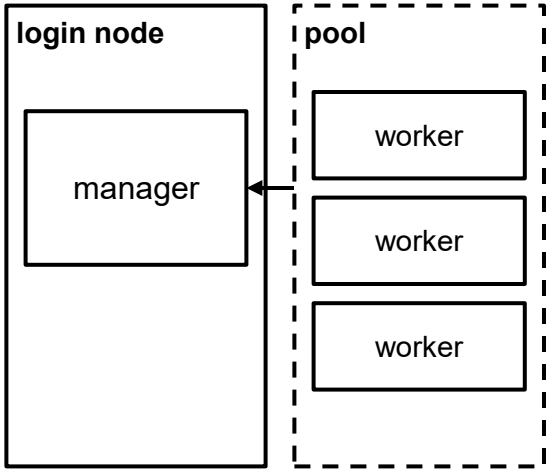
2 Local TaskVine

Module 3



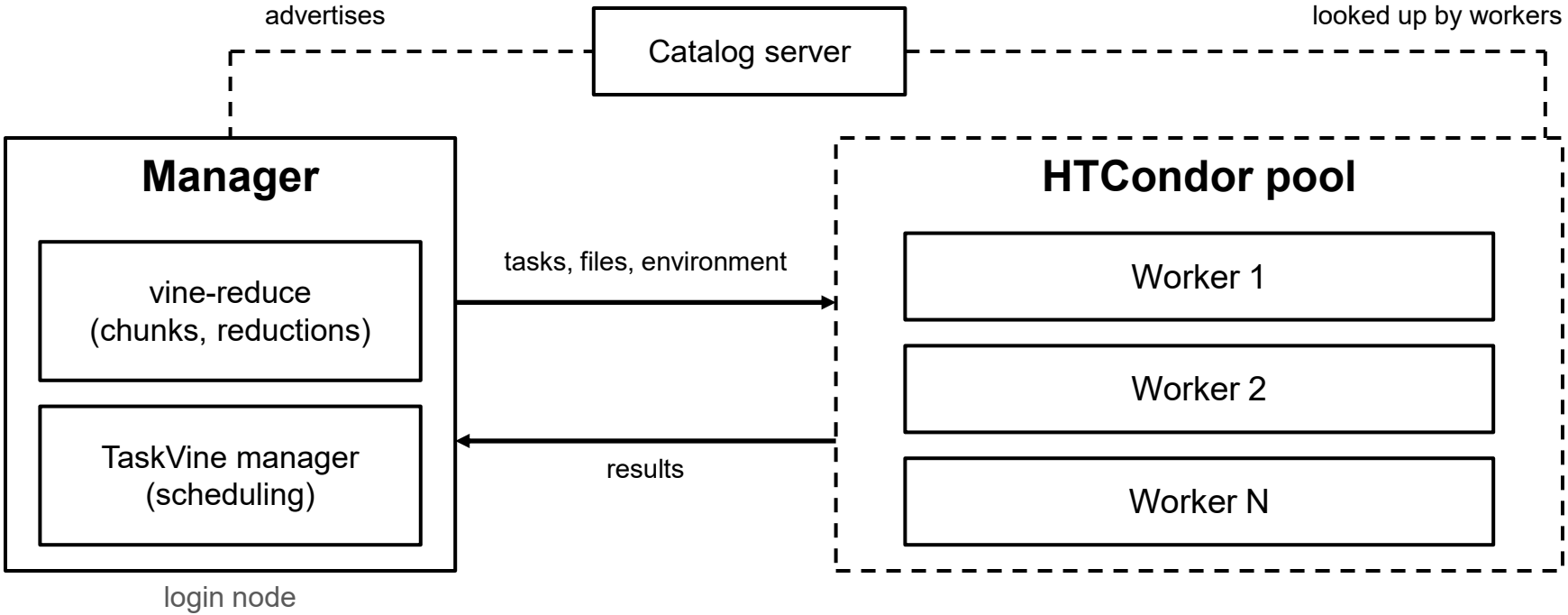
3 HTCondor pool

Module 4



The analysis code is the same in all three. Only what it is handed to changes.

Manager and workers



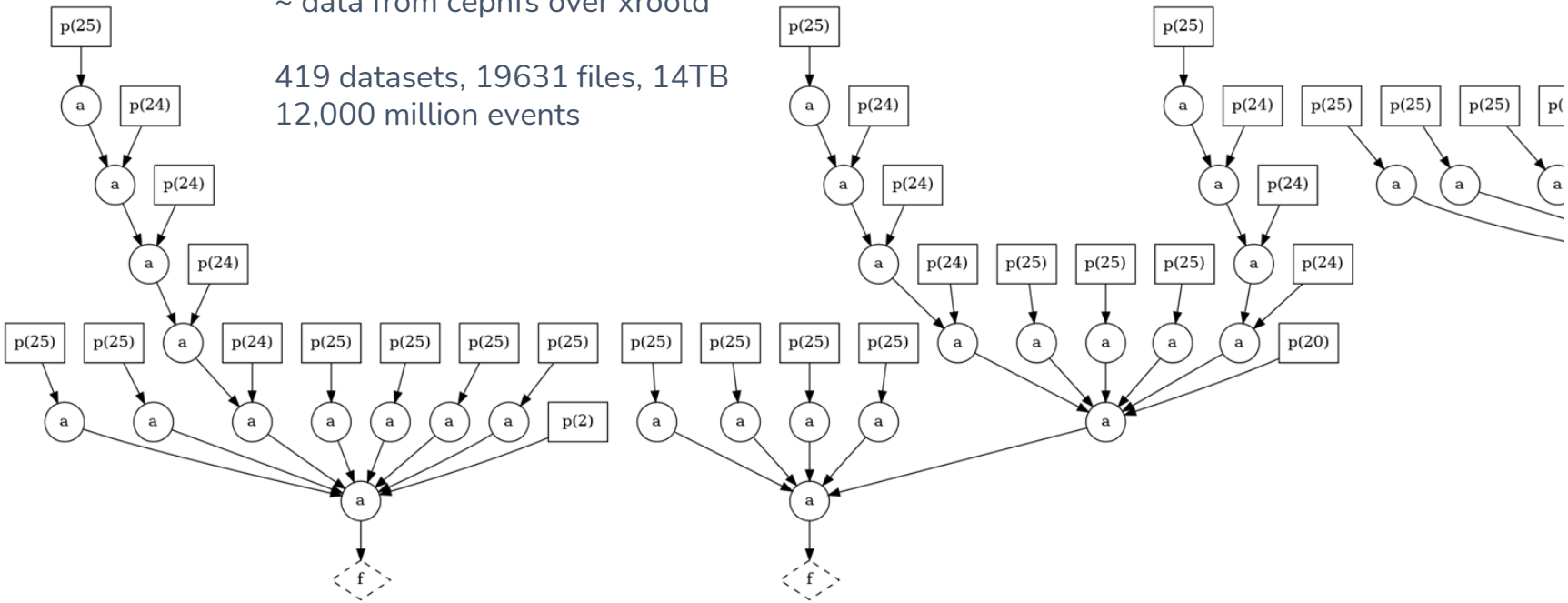
Workers find the manager through the catalog, connect to it, and read input data from storage (xrootd or a shared filesystem).

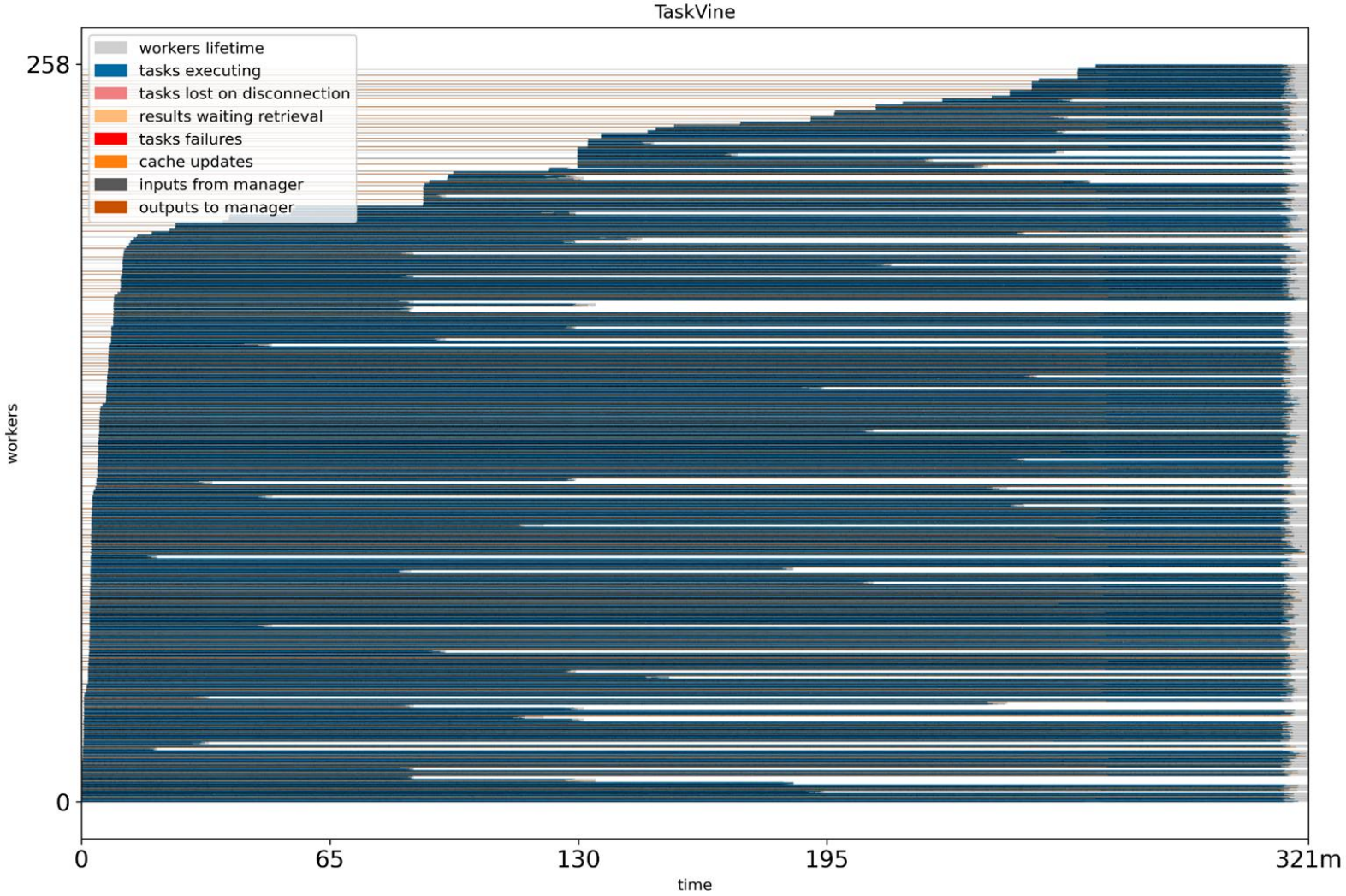
Example: Cortado workflow (Kelci M.)

~ 5.5 hours over 1600 cores
~130k proc taskvine tasks
~ 13k + reduction tasks

~ data from cephfs over xrootd

419 datasets, 19631 files, 14TB
12,000 million events



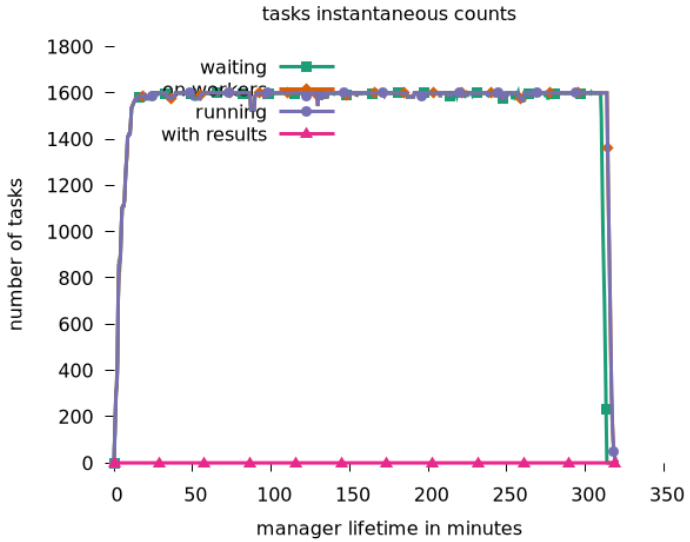


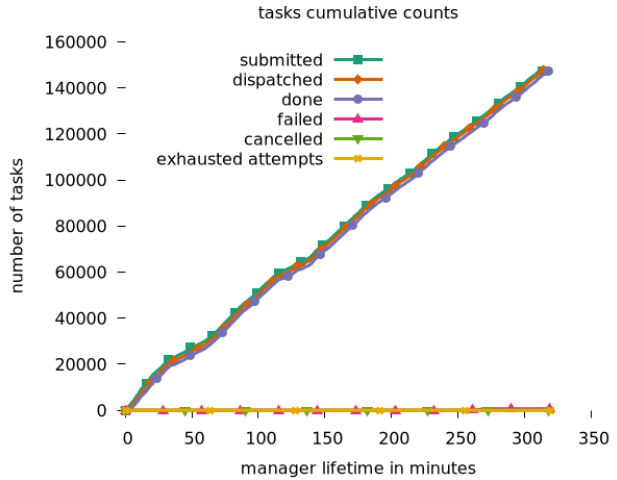
8-core workers

58 workers lost/replaced during execution

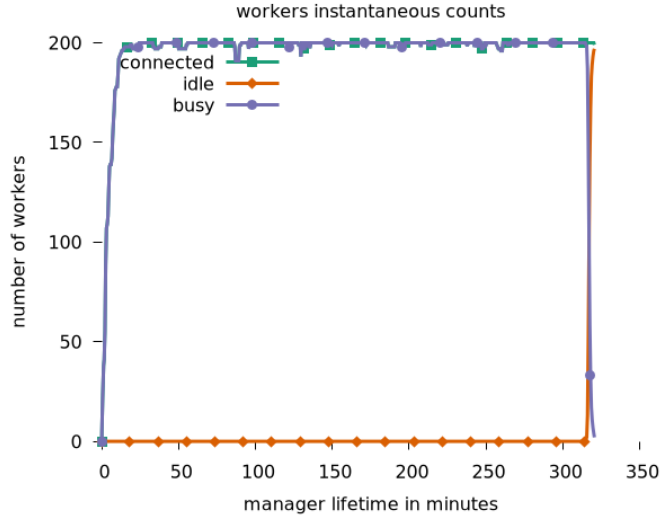
at most 1600 cores at any time

(many xrootd timeouts, figuring out new setup)

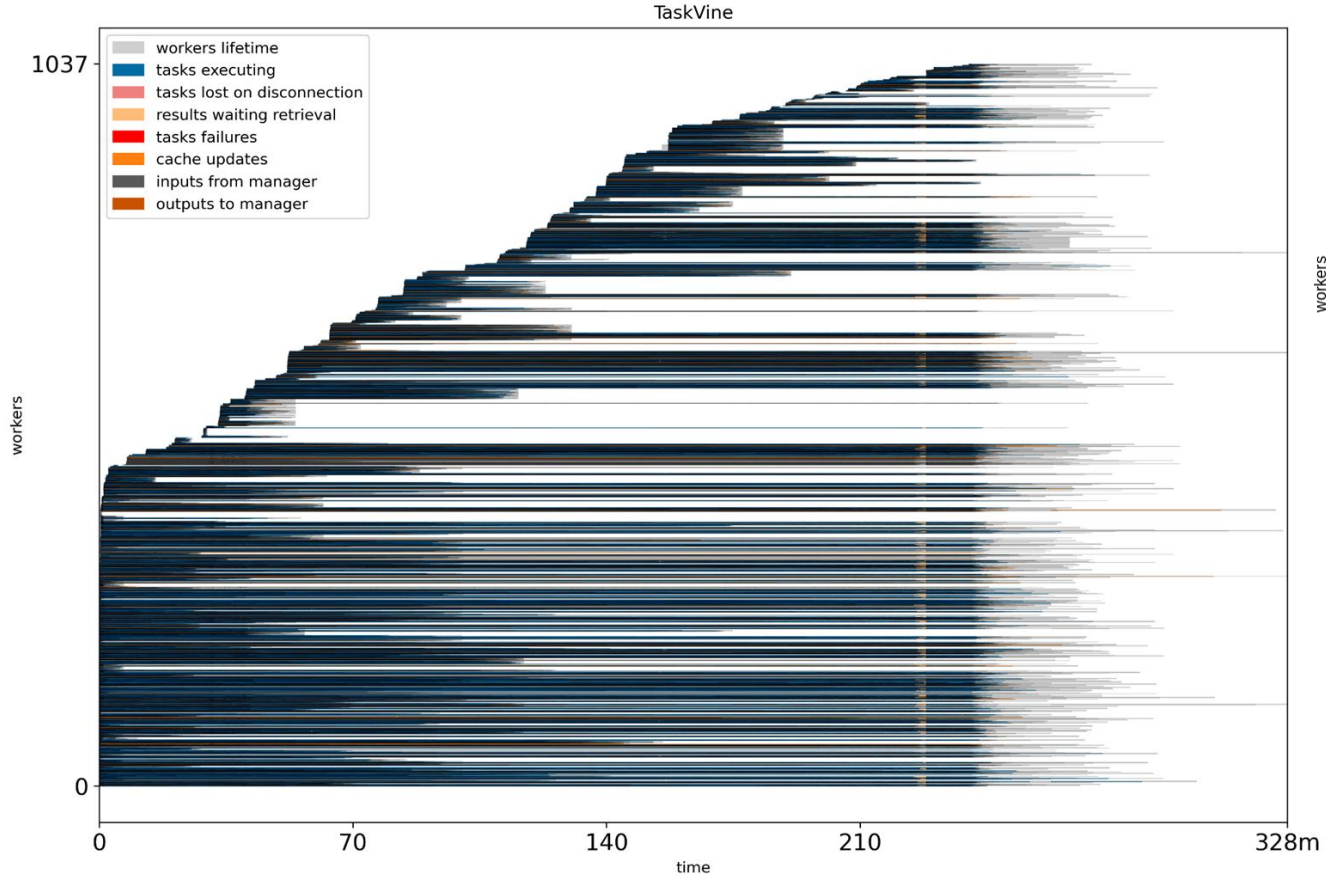

tasks instantaneous counts
waiting
on workers
running
with results
number of tasks
manager lifetime in minutes


tasks cumulative counts
submitted
dispatched
done
failed
cancelled
exhausted attempts
number of tasks
manager lifetime in minutes


manager time proportions
utilization per time sample (%)
manager lifetime in minutes
other
application
receive
send
polling
internal
status msgs


workers instantaneous counts
connected
idle
busy
number of workers
manager lifetime in minutes

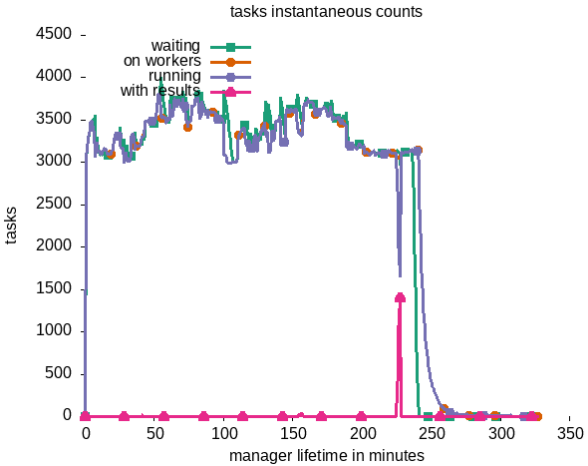
mdv5 (Connor) run. ~110K total tasks

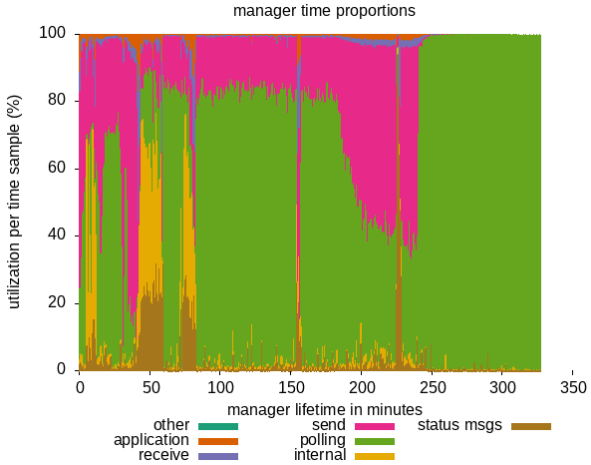


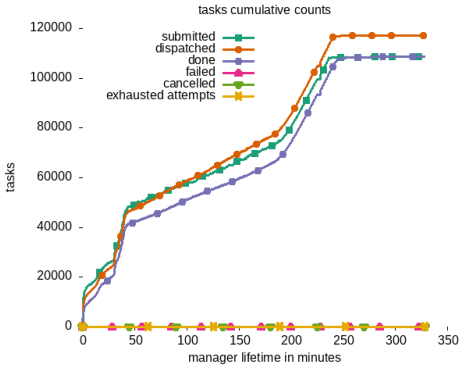


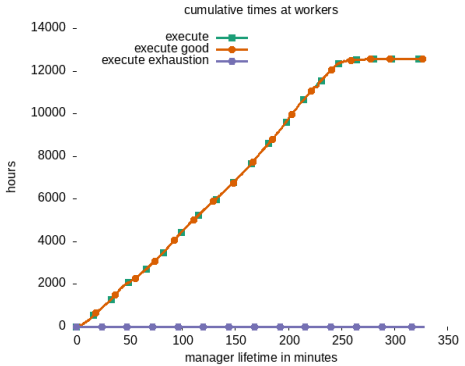
high number of workers replaced

51 datasets
2.6 TB
8130 files (vast)









99960 proc tasks
4111 accum tasks
4478 long proc tasks
checkpointed (> 1800s)
3297 checkpointed

no recovery tasks
file replication: 3

aggressive priority, datasets scheduled together, accum/checkpoint tasks higher priority

manually set chunksize.
10K for 49 ds,
1K for 2 ttboosted_.* ds

What the manager lets you control

Module 5	**Resources and chunk size**	What a task asks for, and how big it is
Module 6	**Checkpoints and restart**	Stop a run, resume without redoing work
Module 7	**Multiple processors**	Several analyses in one run
Module 8	**result_postprocess**	Big outputs bypass the manager
Module 10 (extra)	**failure_proportion**	Keep going when some chunks fail

Where to get help

NDCMS Mattermost

`https://mattermost.web.cern.ch/ndcms/channels/cctools`

Cooperative Computing Lab

`https://cclnd.cse.nd.edu`

TaskVine documentation

`https://cctools.readthedocs.io/en/stable/taskvine`

GitHub

`github.com/cooperative-computing-lab: vine-reduce, vine-cms-analysis-stack, cctools`